

AI Resume Job Recommendation System project report

Prepared By: K.ROJASRI

Programme: B.Tech Information Technology

College: KLN College of Engineering

Academic Year: 2026–2027

A Flask-based web application that analyzes uploaded resumes and recommends suitable jobs using skill matching, NLP similarity and experience matching.

1. Abstract

The AI Resume Job Recommendation System is a web-based application developed using Python and Flask. The system accepts a resume in PDF or DOCX format, extracts relevant information, identifies skills and profile details, and compares the candidate profile with available job opportunities. It provides job recommendations based on skill matching, natural language processing (NLP) similarity and experience compatibility. The project also includes pages for resume analysis, skill-gap information, career insights, dashboards, job details, saved jobs and resume history.

The uploaded project contains a Flask application, HTML templates, two CSV datasets and a testing/analysis script. The recommendation logic in the project combines skill and keyword matching with TF-IDF/cosine similarity and experience scoring to calculate an overall recommendation score.

2. Introduction

Recruiters and job seekers often need to compare large amounts of resume and job information. Manually finding suitable jobs from a resume can take time and may miss relevant opportunities. This project provides an automated approach in which resume information is extracted and compared with a predefined job database.

The application is implemented as a Flask web application. Its interface contains a home page, resume upload page, analysis page, jobs page, job details page, dashboard, profile, resume history, saved jobs, skill gap and career insights pages.

3. Problem Statement

Traditional job searching requires candidates to read many job descriptions and manually compare their skills, education and experience. A system is therefore needed to analyze resume information and provide relevant job suggestions in a simple web interface.

4. Objectives

- To upload and process resumes in PDF/DOCX format.
- To extract useful candidate information such as skills, education, experience and profile details.
- To compare candidate skills with job requirements.
- To use NLP-based text similarity for additional matching.
- To consider experience compatibility in recommendations.
- To display job recommendations and related career information through a web interface.

5. Existing System

In a conventional job search process, candidates search job portals manually, read job

descriptions, compare required skills and shortlist suitable roles. This approach depends heavily on manual effort and does not automatically calculate a candidate-to-job match.

6. Proposed System

The proposed system automates resume-to-job comparison. After a resume is uploaded, the application processes the document, extracts relevant information and calculates matching scores against available jobs. The result is displayed through a Flask web interface with supporting analysis and career pages.

7. System Requirements

Category	Requirement
Web Framework	Flask
Frontend	HTML, CSS, JavaScript/templates
Data Processing	Pandas, NumPy
NLP / ML	Scikit-learn, TF-IDF, Cosine Similarity
Resume Processing	pypdf, python-docx
Data	job_dataset.csv, ml_resume_dataset_4500.csv

8 . Technologies Used

Python: Main programming language used for application and recommendation logic.

Flask: Handles web routes, pages, form submissions, sessions and file uploads. **Pandas:** Used in the test/analysis script for loading and analyzing the CSV datasets. **Scikit-learn:** Used for TF-IDF vectorization and cosine similarity.

pypdf / python-docx: Used for extracting text from PDF and DOCX resumes.

HTML/CSS: Used to build the web pages and user interface.

9.System Architecture

The system follows a simple web application flow:

User □ Flask Web Interface □ Resume Upload □ Resume Text Extraction □ Skill/Profile Analysis □ Job Dataset Comparison □ Matching/NLP/Experience Scoring □ Recommended Jobs □ Dashboard/Analysis Pages

10.System Modules

Resume Upload: Allows the user to upload a resume and start analysis.

Resume Analysis: Displays extracted candidate information and analysis results.

Job Recommendation: Compares the resume with available jobs and produces recommendations.

Skill Gap Analysis: Shows information useful for identifying skills that may need improvement.

Dashboard: Provides a central view of profile and recommendation-

related information. **Job Details:** Displays details such as role, company, location, salary, experience and skills. **Profile:** Provides a profile-oriented view of candidate information.

Resume History: Provides a place for previous resume-related information.

Saved Jobs: Provides a place to keep selected jobs.

Career Insights: Provides career-oriented information based on the application data.

11. Dataset Description

The uploaded project contains two datasets: **job_dataset.csv** and **ml_resume_dataset_4500.csv**. The testing script loads both datasets using Pandas and checks their shapes, columns, missing values, duplicate records and sample fields. Job data includes fields such as job title, skills, keywords and experience level. Resume data includes fields such as skills, current title, years of experience, highest degree and raw resume text.

The test script also performs basic cleaning of resume and job skills, calculates skill-match percentages and evaluates TF-IDF similarity between resume text and job information.

12. Machine Learning / NLP Methodology

Step 1 – Skill Matching: Resume skills and job skills are converted into sets. Common skills are identified and a percentage score is calculated.

Step 2 – Experience Matching: The candidate's years of experience are compared with the job's experience level. A compatibility score is calculated.

Step 3 – NLP Similarity: Important resume and job text is converted into TF-IDF vectors. Cosine similarity measures how similar the resume text is to each job.

Step 4 – Weighted Score: The improved logic combines skill/keyword matching, NLP similarity and experience compatibility. The project code gives the combined skill score the largest weight, followed by NLP similarity and experience score.

Step 5 – Recommendation: Jobs are sorted by the calculated score and the top distinct job titles are selected as recommendations.

13. Implementation

The main application file is **app.py**. It creates the Flask application, configures the upload folder, stores a job database and defines the web application behavior. The project contains multiple HTML templates such as `index.html`, `upload.html`, `analysis.html`, `jobs.html`, `job_details.html`, `dashboard.html`, `profile.html`, `resume_history.html`, `saved_jobs.html`, `skill_gap.html`, `career_insights.html` and `about.html`.

The file **test.py** is used for dataset inspection and recommendation testing. It demonstrates missing-value checks, duplicate checks, skill cleaning, skill matching, TF-IDF similarity and weighted recommendation scoring.

14. Working Procedure

1. Open the Flask project in VS Code.
2. Install the required Python packages.
3. Run the Flask application.
4. Open the local Flask URL in a browser.
5. Upload a PDF or DOCX resume.
6. The application extracts and analyzes resume information.
7. Skills, keywords, NLP similarity and experience are compared with job records.
8. The application displays recommended jobs and supporting analysis.

15. Results / Output

The expected output is a web interface in which a candidate can upload a resume, view analyzed information and receive job recommendations. The recommendation section is based on the matching logic implemented in the project. The job pages can present role information including title, company, location, salary, experience, job type, skills and description.

The project also provides supporting pages for dashboard information, skill gaps, career insights, saved jobs and resume history, making it suitable as a portfolio-style full-stack/AI project.

16. Advantages

- a. Reduces manual effort in comparing resumes with job descriptions.
- b. Combines multiple matching signals rather than relying on a single keyword.
- c. Provides an easy-to-use web interface.
- d. Supports PDF/DOCX resume processing.
- e. Can be extended with larger datasets and more advanced machine-learning models.

17. Limitations

- a. The job information in the project is based on the included datasets/static job records.
- b. Recommendation quality depends on the quality and coverage of the resume and job data.
- c. The current matching approach is not a replacement for human recruitment decisions.
- d. Deployment, authentication, database persistence and production security would require additional implementation.

18. Future Enhancements

- a. Connect to live job APIs or a maintained job database.
- b. Add a trained classification/ranking model using labeled resume-job pairs.
- c. Add user authentication and a production database.
- d. Improve skill extraction using modern NLP models.
- e. Add personalized learning/course recommendations for missing skills.
- f. Deploy the application to a cloud platform with secure file handling.

19. Conclusion

The AI Resume Job Recommendation System demonstrates how a Flask web application can combine resume processing, data analysis and NLP techniques to support job discovery. The uploaded project includes the core application, web templates, datasets and a testing script. Its recommendation logic combines skill and keyword matching, TF-IDF similarity and experience compatibility to produce job suggestions. The project can serve as a practical academic and GitHub portfolio project and can be expanded into a more scalable production system.

20. References

- 1. Python documentation and Flask documentation.
- 2. Scikit-learn documentation for TF-IDF and cosine similarity.
- 3. Pandas documentation for data processing.
- 4. pypdf and python-docx documentation for document text extraction.